

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
SIMATS ENGINEERING
COMPUTER SCIENCE AND ENGINEERING
CSA17 - ARTIFICIAL INTELLIGENCE

Experiment No. 2: Python Program to Solve the 8-Queen Problem

Aim

To write a Python program to solve the 8-Queen problem, which involves placing 8 queens on an 8x8 chessboard such that no two queens attack each other (i.e., no two queens share the same row, column, or diagonal), using the Backtracking technique.

Algorithm

1. Start.
2. Represent the chessboard as a 1-D list 'board' of size N (N = 8), where board[row] stores the column position of the queen placed in that row.
3. Define a function is_safe(board, row, col, n) that checks whether placing a queen at (row, col) conflicts with any previously placed queen — same column or same diagonal.
4. Define a recursive function solve_n_queens(board, row, n, solutions) that attempts to place a queen in every row one at a time.
5. For the current row, try placing the queen in each column from 0 to N-1.
6. For each column, call is_safe(); if safe, place the queen (board[row] = col) and recursively attempt to solve for the next row.
7. If row equals N, a valid arrangement of all N queens has been found — store this configuration in the solutions list.
8. After the recursive call returns, remove the queen (backtrack) by resetting board[row] = -1, and try the next column.
9. Repeat the process until all columns for the current row have been tried.
10. Continue until all possible safe configurations have been explored.
11. Print the total number of solutions found and display the first valid board configuration.
12. Stop.

Pseudocode

```
BEGIN
  INPUT: n = 8
  board = array of size n, initialized to -1
  solutions = empty list

  FUNCTION is_safe(board, row, col, n)
    FOR i = 0 TO row - 1 DO
      IF board[i] == col OR |board[i] - col| == |i - row| THEN
        RETURN false
      END IF
    END FOR
    RETURN true
  END FUNCTION
```

```

FUNCTION solve(board, row, n)
  IF row == n THEN
    ADD copy of board to solutions
    RETURN
  END IF

  FOR col = 0 TO n - 1 DO
    IF is_safe(board, row, col, n) THEN
      board[row] = col
      solve(board, row + 1, n)
      board[row] = -1  // backtrack
    END IF
  END FOR
END FUNCTION

CALL solve(board, 0, n)
PRINT total number of solutions
PRINT first solution board
END

```

Result

The Python program for the 8-Queen Problem was executed successfully using the Backtracking algorithm. The program systematically placed queens row by row, checking safety conditions and backtracking whenever a conflict occurred. All 92 valid solutions for placing 8 non-attacking queens on an 8x8 chessboard were found, and the first valid configuration was displayed. Thus, the 8-Queen problem was solved successfully using Python.

Output Screenshot:

```

Terminal - 8queens.py

$ python3 puzzle8queens.py
Total solutions found for 8-Queens problem: 92

First solution:

Q . . . . . .
. . . . Q . .
. . . . . . Q
. . . . . Q .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .

Queen positions (row -> column): [0, 4, 7, 5, 2, 6, 1, 3]

```